



Name: Shaikh Mohammed Abbas Pareez Enroll: 23111590423

Class: CDSK (B)

Subject: OSY

Assignment 2

1) Explain round robin algorithm with suitable example.

→ Round robin algorithm is a strictly preemptive scheduling algorithm.

A small unit of time known as a time quantum or time slice is used for pre-emption of a currently running process. CPU is assigned to the entire processes one by one on first come first serve basis for the specified time quantum.

Every process executes for the specified time quantum, after which the dispatcher is invoked to pre-empt current running process and CPU is given to the next process.

A new process is added at the tail of the ready queue when it enters the system.

If the burst time of a process is ^{greater} ~~less~~ than time quantum, then context switch occurs and the process is placed at tail for further execution.

If the burst time of a process is less than time quantum, the process itself releases the CPU.

Example:

Process	Burst time (BT)	Arrival time (AT)
P1	24	0
P2	3	0
P3	3	0

Time quantum = 4 ms

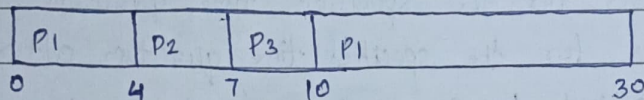
Formulae:

Turnaround time (TAT) = Completion time (CT) - AT

Waiting time (WT) = TAT - BT

Process	AT	BT	CT	TAT	WT
P1	0	24	30	30	6
P2	0	3	7	7	4
P3	0	3	10	10	7

Gantt chart:



$$\text{Avg. TAT} = \frac{30+7+10}{3} = \frac{47}{3} = 15.67\text{ms}$$

$$\text{Avg. WT} = \frac{6+4+7}{3} = \frac{17}{3} = 5.67\text{ms}$$

2) Difference between preemptive and non-preemptive scheduling algorithm

→	Preemptive scheduling	Non-preemptive scheduling
i)	Even if CPU is allocated to one process, CPU can be preempted to other process if other process is having higher priority or some other fulfilling criteria	Once, the CPU has been allocated to a process, the process holds the CPU until releases CPU either by terminating or by switching to waiting state.
ii)	It has high overhead.	It has low overhead.



Preemptive scheduling	Non-preemptive scheduling
iii) Throughput is less	Throughput is high
iv) Algorithm design is complex	Algorithm design is simple.

3) What is deadlock?

→ A deadlock is a situation where a set of processes are blocked because each process is being holding a resource and waiting for another resource acquired by some other process.

4) State the necessary conditions for deadlock.

→ A deadlock is a situation where a set of processes are blocked because each process is being holding a resource and waiting for another resource acquired by some other process.

Following are the necessary conditions for deadlock:

1) Mutual exclusion:

The mutual exclusion condition must hold for non-sharable resources.

Sharable resources do not require mutually exclusive access, thus cannot be involved in a deadlock.

2) Hold and Wait:

~~One way to avoid A process must be holding atleast one resource and waiting to acquire additional resources that are currently being held by other processes.~~

3) No pre-emption:

Resources cannot be pre-empted, i.e., a resource can be released only voluntarily by the process holding it.

4. Circular wait:

A set $\{P_0, P_1, \dots, P_n\}$ of waiting processes must exist such that P_0 is waiting for a resource held by P_1 , P_1 is waiting for a resource held by P_2 , ..., P_{n-1} is waiting for the resources held by other waiting processes in ~~circle~~ P_n and P_n is waiting for a resource held by P_0 .

Each process is waiting for the resources held by other waiting processes in circular form.

5) Explain how deadlock can be prevented.

→ By ensuring that at least one of below conditions cannot hold, we can prevent the occurrence of a deadlock.

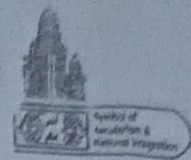
1) Mutual exclusion:

The mutual-exclusion condition must hold for non-sharable resources. Sharable resources do not require mutually exclusive access, thus cannot be involved in a deadlock.

2) Hold and Wait:

One way to avoid this Hold and Wait is when a process requests a resource; it does not hold any other resources.

- One protocol that can be used requires each process to request and be allocated all its resources before it begins execution.
- Another protocol that can be used is, to allow a process to request resources only when the process has none. A process may request some resources and use them. Before it requests any additional resources, it must release all resources that are currently allocated to it.



3) No preemption:

If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are preempted. That is these resources are implicitly released. The preempted resources are added to the list of resources for which the process is waiting. Process will be restarted only when all the required resources will be available.

4) Circular wait:

Circular-wait condition never holds is to impose a total ordering of all resource types, and to require that each process requests resources in an increasing order of enumeration.

Let $R = \{R_1, R_2, \dots, R_n\}$ be the set of resources types.

We assign to each resource type a unique integer number, which allows us to compare two resources and to determine whether one precedes another in our ordering.

Formally, define a one-to-one function $F: R \rightarrow N$, where N is the set of natural numbers.

6) Explain partitioning in detail.

→ An important operation of memory management is to bring programs into main memory for execution by the processor.

Partitioning is a technique that divides a memory into multiple partitions.

These partitions can be of different size or same size.

Types of partitioning:

- Fixed partitioning, i.e., static partitioning.
- Variable partitioning, i.e., dynamic partitioning.

Fixed partitioning:

Main memory is divided into multiple partitions of fixed size at the time of system generation.

A process may be loaded into a partition of equal or greater size.

There are two types of fixed partitioning:

- 1) Equal sized partitioning.
- 2) Unequal sized partitioning.

Operating system 8M
8M
8M
8M
8M
8M
8M
8M

Fig: Equal fixed size partitioning.

operating system 8M
2M
4M
6M
8M
8M
12M

Fig: Unequal fixed size partitioning.

Variable partitioning:

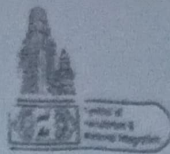
When a process enters in main memory, it is allocated exact size that is required by that process.

So in this method, partitions can vary in size depending on memory space required by a process entering in main memory.

Operating system maintains a table indicating which parts of memory are available and which are occupied.

When new process arrives and it needs space, system searches for available memory space in main memory.

If it is available, then memory is allocated to the process by



creating a partition in memory.

Example:

Process	Memory Space
P1	20M
P2	14M
P3	18M

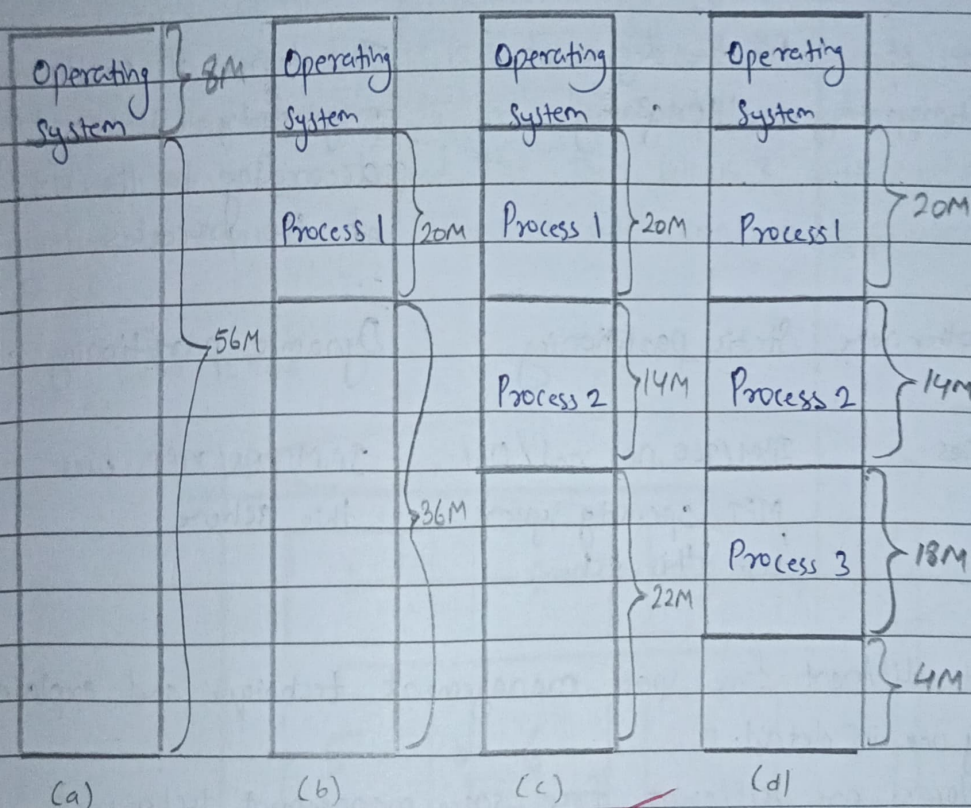


Fig: Variable size partitioning.

7) Differentiate between fixed and variable partitioning.

Basis	Fixed size partitioning	Variable size partitioning.
Fragmentation	It supports internal fragmentation.	It supports external fragmentation.

Basis	Fixed-size Partition	Variable-size partition
Size of partition	OS decides the size of the partition only once during the system boot time. It can't be changed.	OS decides the size of the partition according to the size of partition process. It can be changed.
Degree of multiprogramming	Fix the degree of multiprogramming.	The degree of multi-programming will vary according to the size of the processes.
Another name	Static partitioning	Dynamic partitioning
Uses	IBM/360, DOS and OS/ MFT operating systems use this scheme.	IBM, OS/ MFT uses this scheme.

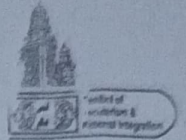
8) List different free space management technique and explain any one in detail.

→ Following are different free space management techniques:

- 1) Bitmap
- 2) Linked List
- 3) Grouping

Linked List:

Another method of doing free space management in operating systems is a linked list, in which all the free blocks existing in the disk are linked together in a linked list.



The address of the first free block is stored somewhere in the memory.

Each free block contains a pointer that contains the address to the next free block.

The last free block points to null, indicating the end of the linked list.

For example, consider a disk having 16 blocks where block numbers 3, 4, 5, 6, 9, 10, 11, 12, 13 and 14 are free, and the rest of the blocks, i.e., 1, 2, 7, 8, 15, 16 are allocated to some files.

If we maintain a linked list, then Block 3 will contain a file pointer to Block 4 and Block 4 will contain a pointer to Block 5.

Linked list head

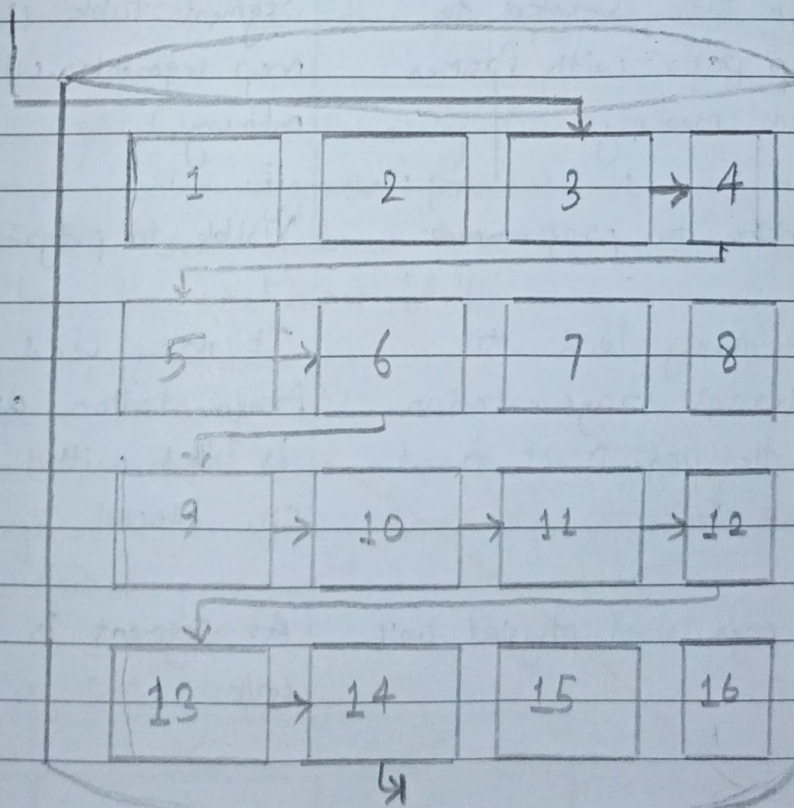
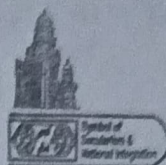


Fig: Linked list free space management technique.

9) Difference between paging and segmentation.

→ Paging	Segmentation
i) It divides the physical memory into frames and program's address space into same size pages.	It divides the Computer's physical memory and program's address space into segments.
ii) Page is always of fixed block size.	Segment is of variable size.
iii) The size of the page is specified by hardware.	The size of the segment is specified by the user.
iv) Page table is used to map pages with frames from memory.	Segment table is used to map segments with physical memory.
v) Invisible to programmer.	Visible to programmer.
vi) It may lead to internal fragmentation as the page is of fixed block size.	It may lead to external fragmentation as the memory is filled with the variable size blocks.
vii) A page is of physical unit.	As segment is of logical unit.



10) With suitable example explain preemptive and non-preemptive SJF algorithm.

→ In this algorithm, the CPU is assigned to a process that has smallest next CPU burst.

If the next CPU burst of two processes is same, FCFS scheduling is used to break the tie.

It is called the shortest next CPU burst algorithm because scheduling depends on the length of the next CPU burst of a process.

With the SJF scheduling algorithm, the average waiting time is minimum, because it moves short processes before long processes.

This decreases the waiting time for short processes more than increases the waiting time for long processes.

This algorithm can be either preemptive or non-preemptive.

Preemptive SJF (Shortest Remaining Time Next):

The preemptive SJF algorithm preempts the currently running process if the remaining burst time of the current process is greater than the CPU burst of the newly arrived process.

Non-preemptive SJF:

This algorithm allows the currently running process to continue even though the remaining burst time is more than that of the newly arrived process.

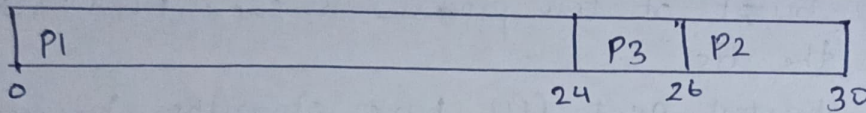
Considering the following example:

Process	Burst time	Arrival time
P1	24	0
P2	4	1
P3	2	2

* Non-preemptive SJF:

Process	Burst time	Arrival time	Completion time	TAT	WT
P1	24	0	24	24	0
P2	4	1	30	29	25
P3	2	2	26	24	22

Gantt chart:

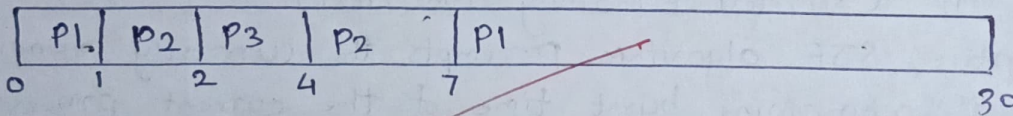


$$\text{Avg. TAT} = \frac{24+29+24}{3} = 25.67 \text{ ms} \quad \text{Avg. WT} = \frac{0+25+22}{3} = 15.67 \text{ ms}$$

* Preemptive SJF:

Process	Burst time	Arrival time	CT	TAT	WT
P1	24	0	30	30	6
P2	4	1	7	6	2
P3	2	2	4	2	0

Gantt chart:



$$\text{Avg. TAT} = \frac{30+6+2}{3} = 12.67 \text{ ms}$$

$$\text{Avg. WT} = 2.67 \text{ ms} = \frac{6+2+0}{3}$$

Handwritten signature/initials